Eidgenössische Technische Hochschule Zürich

Programming Techniques for Scientific Simulations

Autumn Semester 2026

Lecture Notes

Contents

1 Version Control and Git **1**

2 Preprocessing, Compiling, Linking **1**

3 Make **3**

4 CMake **3**

5 Templates and Generic Programming **4**

6 Error Handling **6**

7 Standard Library **7**

References

[1] Käppeli, Roger: Vorlesungsskript PT1. ETH Zürich. 2026

Programming Techniques for Scientific Simulations

Fynn Krebser–fkrebser@student.ethz.ch

Autumn Semester 2026

Lec 1

1 Version Control and Git

When working on a project in a group, it is important to have a system that allows you to track changes to your code and collaborate with others. Version control systems (VCS) are tools that help manage changes to source code over time. They can be local on one computer, centralized, with one centralized repository, or decentralized, with many repositories that can be synced. An example for the last one is git, which we will discuss in this course.

These tools can be used via command line tools, GUIs or via editor plugins. We'll focus on the CLI tools. The general idea is that you have your files and a version database. Every time you are happy with your changes, you can COMMIT them with a short description describing the changes (COMMIT MESSAGE).

Version control is also useful for working in a group. For example Alice and Bob are working on a project. Alice makes some changes, commits and PUSHes them to a remote repository. Later on, Bob can PULL the changes from the remote repository and continue working on the project.

Usually however, both peoples work together. Alice commits and pushes her changes, while Bob is still working. He commits his new version and tries to push it. However, this time, the push can fail, because Alice's changes are not yet in Bob's local repository. This is a CONFLICT. We also say that the histories have diverged. What Bob needs to do is MERGING. Bob inspects both versions and decides how to combine them. After that, he can commit the merged version and push it to the remote repository.

In git, we can init a repository with `git init`. This creates a new repository in the current directory. The information for this is stored in a hidden directory called `.git`. With `git status` we can check the status of our repository. It will show us which files have been modified, added or deleted. We can add files to the staging area with `git add <file>`. This means that we are ready to commit these changes. Finally, we can commit the changes with `git commit -m "commit message"`. The commit message should be a short description of the changes made.

The STAGING AREA exists, so you have some freedom to decide which changes you want to commit. You can add some files to the staging area, but not others. This is useful if you are working on multiple features at the same time and want to commit them separately. Using `git diff` we can see the changes between the working directory and the staging area. With `git diff -staged` we can see the changes between the staging area and the last commit.

Usually in a repository we only want the source code and not some artifacts or executables. We can tell git to ignore certain files by creating a file called `.gitignore`. In this file we can specify patterns for files that should be ignored by git. For example, we can ignore all files with the extension `.log` by adding the line `*.log` to the `.gitignore` file.

Sometimes it could be useful to work on two different features. In this case we can use BRANCHES. A branch is a separate line of development. We can create a new branch with `git branch <branch name>`. We can switch to a different branch with `git checkout <branch name>`. After having made some changes and committed them, we can switch back to the main branch and notice, there are no changes. Once we are happy with the new branch, we can merge it back into the main branch with `git merge <branch name>`. If we are done, we can delete the branch with `git branch -d <branch name>`.

If there is a merge conflict, git will mark the files that have conflicts and we need to resolve them manually. After resolving the conflicts, we can add the files to the staging area and commit the changes.

Lec 2

2 Preprocessing, Compiling, Linking

A compiler is a program that translates source code into machine code. We will mainly use GNU and Clang. When compiling a program, the PREPROCESSOR is the first step. It manipulates the source code. For this all commands start with a `#`. For this, we can define macros with `#define`. For example, we can define a macro `PI` as follows:

```
#define PI 3.14159265358979323846
```

The convention for macros is to use all uppercase letters. With this one can distinguish between macros and functions defined in the source code. Macros can also have arguments. For example

```
#define SUM(A,B) A + B
```

In the console this can also be done with the flag `-D`. For example, we can define a macro `PI` as follows:

```
g++ -DPI=3.14159265358979323846 main.cpp
```

We can also undefine macros with `#undef` and in the console with the flag `-U`. For example, we can undefine a macro `PI` as follows:

```
#undef PI
```

To look at only the preprocessed output is to do

```
g++ -E main.cpp
```

The preprocessor also allows us to include other files with `#include`. For example, we can include the header file `iostream`.

As a warning, the preprocessor does not check for syntax errors. It simply replaces the macros and includes the files. Therefore, it is important to be careful when using macros and includes.

For conditional compilation, we can use `#ifdef`.

```
#ifdef PI
    std::cout << "PI is defined" << std::endl;
#else
    std::cout << "PI is not defined" << std::endl
    ;
#endif
```

When including files, we can include them with brackets or with quotes. The brackets are used for system headers, while the quotes are used for user-defined headers. With the `-I` flag, we can tell the compiler where to look for the header files.

The next step is the Compiler. It turns the preprocessed code into object code. We can view the object code with the `-S` flag. For example, we can compile a file `main.cpp` into an object file `main.o` as follows:

```
g++ -c main.cpp -o main.o
```

Furthermore we can change the optimization level with the `-O` flag. The optimization levels are from 0 to 3. The higher the optimization level, the more time it takes to compile, but the faster the program will run. The default optimization level is 0. For example, we can compile a file `main.cpp` with optimization level 2 as follows:

```
g++ -O2 main.cpp -o main
```

After that, the assembler takes the object code and turns it into machine code. The machine code is then linked with other object files and libraries to create an executable file.

To see how linking works, let us consider some code refactoring. In general, Programms can be split into several files, that can be compiled separately and finally lined together. For this in a .hpp file we put in the function declarations and in a .cpp file we put in the function definitions. For example, we can have a file `math.hpp` with the following content:

```
double add(double, double);
double subtract(double, double);
```

And a file `math.cpp` with the following content:

```
#include "math.hpp"
double add(double a, double b) {
    return a + b;
}
double subtract(double a, double b) {
    return a - b;
}
```

Usually we include the hpp file instead of the cpp file. This is done to enforce the consistency between the declaration and the definition. If we include the cpp file, we can have a situation where the declaration and the definition are not consistent. For example, if we change the declaration in the hpp file, but forget to change the definition in the cpp file, we will get a linker error.

To compile all files seperately we compile the files with the `-c` flag. For example, we can compile the files `math.cpp` and `main.cpp` as follows:

```
g++ -c math.cpp -o math.o
g++ -c main.cpp -o main.o
```

After that we can link them with

```
g++ math.o main.o -o main
```

One thing to keep in mind is that when including headers, we should use include guards to prevent multiple inclusions of the same header file. This can be done with the following pattern:

```
#ifndef MATH_HPP
#define MATH_HPP
double add(double, double);
double subtract(double, double);
#endif
```

Alternatively one can also use `#pragma once` at the top of the header file. This is a non-standard but widely supported way to prevent multiple inclusions of the same header file.

```
#pragma once
double add(double, double);
double subtract(double, double);
```

One useful macro might be `NDEBUG`. If this macro is defined, the `assert` macro will not check for the condition and will not terminate the program if the condition is false. This can be useful for performance reasons, as the `assert` macro can add overhead to the program. However, it is important to note that this can also lead to undefined behavior if the condition is false and the program continues to run.

Libraries come in two kinds. First, static libraries aka archives. When linking, only the used functions are linked into the executable. Second, dynamic libraries aka shared objects. When linking, all functions are linked into the executable. The dynamic libraries are loaded at runtime. This means that if a function is not used, it will not be loaded into memory. This can save memory and reduce the size of the executable. However, it also means that if a function is used, it will be loaded into memory and will take up space.

We can pack the `*.o` files into a static library with the `ar` command. For example, we can create a static library `libmath.a` from the object files `math.o` and `main.o` as follows:

```
ar -crs libmath.a math.o main.o
```

Important the lib prefix is required for static libraries. Otherwise, the linker will not be able to understand it.

For shared libraries, we use

```
    c++ -shared -fPIC -o libmath.so math.o main.o
```

The advantage of shared libraries is that the executables are shared between different programs. Thus it might suffice to only have one copy of cout for example.

We now compile and link the program with the shared library as follows:

```
c++ -c -Ilib main.cpp
c++ -o main main.o -Llib -lmath
```

Here -L specifies the path to the library and -l specifies the name of the library. The linker will look for a file named `libmath.so` in the specified path. -I specifies the path to the header files. The linker will look for a file named `math.hpp` in the specified path.

Warning, the order in which libraries are specified is important. The linker will look for the libraries in the order they are specified. If a library is specified before another library that depends on it, the linker will not be able to find the symbols in the first library and will throw an error.

After having made a library, we can document it. For this we should document:

- Synopsis
- Semantics
- Template Parameters
- Parameters
- Return Value
- Precons
- Postcons
- Dependencies
- Exception Guarantees
- Complexity
- References

Unfortunately, after a while, it might become tedious to compile and link many different files. We need to keep track of who depends on whom. Or, we could use build systems.

3 Make

Make is a build automation tool. It is probably the most widely used build tool. Make builds targets based on so-called build files. Usually, the build file is named `Makefile`. Comments are made with hashtags. A simple program might be

```
hello:
    echo Hello, World!
```

Another useful thing might be a dry run to see what would be done without actually doing it. This can be done with the `-n` flag.

For our math library, we can create a Makefile as follows:

```
math.o: math.cpp math.hpp
    c++ -c math.cpp -o math.o

main.o: main.cpp math.hpp
    c++ -c main.cpp -o main.o

main: math.o main.o
    c++ -o main main.o math.o
```

However, make only does the first target. What we can do is a phony target. A phony target is a target that is not a file. It is used to group other targets together. For example, we can create a phony target `all` that builds all targets as follows:

```
.PHONY: all
all: main
```

Make here is pretty smart and will only rebuild the targets that have changed. It does this by checking the timestamps of the files. If a file has changed, it will rebuild the target that depends on it. If a file has not changed, it will not rebuild the target that depends on it.

Lec 3

Make also has automatic variables, which are set after a rule is matched. For example `$@` is the name of the target. Furthermore `$<` is the name of the first prerequisite, and `$^` is the list of all prerequisites.

Make also has some predefined variables. For example CXX is the name of the C++ compiler, CXXFLAGS is the list of flags passed to the C++ compiler, and LDFLAGS is the list of flags passed to the linker. You can use these variables in your Makefile to customize the build process. So we can for example write a rule like this:

```
CXX = c++
CXXFLAGS = -std=c++11
CXXFLAGS += -Wall -Wextra -pedantic

square.o: square.cpp square.hpp
    $(CXX) $(CXXFLAGS) -c $<
```

Something freindly to do is to make a clean target, which removes all build artifacts.

```
.PHONY: clean
clean:
    ${RM} *.o square
```

Since compiler flags are a bit personal, one can also use a config file to set them and include them with `include config.mk`. Putting a minus before, ignores the file if it doesnt exist. The config.mk should also not be under version control.

4 CMake

One annoying thing with make is, that they usually target one system. CMake is a cross platform build system generator. By default CMake generates Makefiles. It can also be used to generate for example Visual Studio project files. To compile our square example, we can create a file called `CMakeLists.txt` with the following content:

```
cmake_minimum_required(VERSION 3.25)
#name of project
project(square)

add_executable(square main.cpp square.cpp)

install(TARGETS square DESTINATION bin)
```

To use CMake, we can use it in the CLI like this:

```
mkdir build
cd build
cmake ..
```

Note, that the build directory can be named however we like. Now we can use the makefile generated by CMake to build the project. To compile more quickly, we can use

```
cmake -S . -B build
```

To create and use a *static* library, we can use

```
add_library(squareLib STATIC square.cpp)

target_include_directories(squareLib PUBLIC ./)

install(TARGETS squareLib
    ARCHIVE DESTINATION lib
    LIBRARY DESTINATION lib
    RUNTIME DESTINATION bin
    )

install(FILES square.hpp DESTINATION include)
```

Then we can link the library to our executable with

```
cmake_minimum_required(VERSION 3.25)
project(square)
add_executable(square main.cpp)
target_link_libraries(square PRIVATE squareLib)
```

5 Templates and Generic Programming

In C++ algorithms are usually very generic. In C++ we can declare a function to have C style nameing by using

```
extern "C" void myFunction();
```

One way to resolve this is by defining a macro. However, there are some problems. For example there is no type safety, its clumsy for longer functions and there can be unexpected side effects.

In C++ we can make use of templates instead.

```
template <typename T>
T min(T x, T y) {
    return (x < y) ? x : y;
}
```

The template only comes to life when it is instantiated. For example, if we call `min(1, 2)`, the compiler will generate a version of the function for `int`. Note that in order to generate a template, the compiler needs to see both the definition (not just declaration) and the instantiation. This is why templates are usually defined in header files.

The good thing about generic programming is that it allows us to write code once and use it for many different types. In general, we identify useful and efficient algorithms, find their generic representation, derive a set of requirements that allow this to run and then implement the algorithm in a generic way.

For the minimum requirements, we need to first be able to copy the type since we pass by value. Furthermore, there must be an operator $<$ defined for the type which result is convertible to `bool`. The first problem can be avoided by passing by reference. Since we dont change the values, we can pass by const reference.

What if we want to call `min` with two different types? For example, `min(1, 2.0)`. This will not work since the compiler will not be able to deduce the type. We can solve this by using two template parameters. However, this will not work either since the return type is ambiguous. We can solve this by using a trailing return type.

```
template <typename R, typename T, typename U>
R min(T x, U y) {
    return (x < y) ? x : y;
}
```

Function templates can have type parameters, non-type parameters and template template parameters. Non-type parameters are usually used for sizes of arrays. Template template parameters are used to pass a template as a parameter to another template. For example, we can define a function that takes a container as a parameter and returns the size of the container.

```
    template <int N> int addN(int val) {return
    val + N;}
```

To do a template specialization, we can use the following syntax:

```
template <>
std::string const& min<std::string const&>(std::
    string const& x, std::string const& y) {
    return (x.length() < y.length()) ? x : y;
}
```

Templates provide direct support for generic programming. It is however only a blueprint. A template puts requirements for its argument. They can be stated in code since C++20 using concepts.

Templates are also type-safe. Objects that are misused will result in a compile error.

Using concepts, we can require ordering of the type by using the following syntax:

```
template <typename T>
concept Ordered = requires(T const& a, T const& b
    ) {
    { a < b } -> std::convertible_to<bool>;
};

template <typename T>
   requires Ordered<T>
T min(T const& x, T const& y) {
    return (x < y) ? x : y;
}
```

Lec 4

As a quick review on classes, they serve to create new data types. I.e. we can extend the data with our own data structures. It allows us to do object oriented programming. Instead of asking what subroutines we need, we can ask what are the abstract entities, what are their properties and how can these be manipulated.

The principles behind this are abstraction, encapsulation, inheritance and polymorphism.

Abstraction is the high level of abstraction that is possbile, encapsulation is the ability to hide representation dependent detail, inheritance is the ability to create new classes based on existing classes, and polymorphism is the ability to use a single interface for different underlying forms (data types).

Classes are collections of members representing one entity. Members can be Functions, data and types. The members can be public, private or protected. Public members can be accessed from anywhere, it should not be modified later. Private members can only be accessed from within the class, and protected members can be accessed from within

the class and from derived classes. This should only be used for member functions and not data.

Function members can be declared inside the class but be defined outside with the scope operator (::). One advantage of this is that we can separate the files into header and source files.

To define a recurring type in a class with typedef

```
class Point {
  public:
    typedef double value_t;
    value_t x() const { return x_; }
    value_t y()
};

Point::value_t Point::x() const { return x_; }
```

Since C++ one can also use `using` instead of `typedef`.

With classes, we can also overload operators. For this we can for example for complex numbers overload the addition operator

```
class Complex {
    public:
        Complex operator+(const Complex& rhs)
    const {
            return Complex(real_ + rhs.real_,
    imag_ + rhs.imag_);
        }
    private:
        double real_;
        double imag_;
};
```

We can also define the operator as a non member function, but then we need a friend declaration

```
class Complex {
    public:
        friend Complex operator+(const Complex&
    lhs, const Complex& rhs) {
            return Complex(lhs.real_ + rhs.real_,
     lhs.imag_ + rhs.imag_);
        }
    private:
        double real_;
        double imag_;
};
```

One nice thing is to overload the insertion operator « for outputting to streams.

```
std::ostream& operator<<(std::ostream& out, Point
     const& p) {
    out << "(" << p.x() << ", " << p.y() << ")";
    return out;
}
```

In C++ it is not possible to have local functions. However we can use function objects. For example we can do

```
Potential V;
double distance;
std::cout << V(distance) << std::endl;
```

Here, we have to overload the function call operator () in the class Potential. Since C++ 11, we can also use lambda functions. The syntax is

```
auto f = [](double x) { return x*x; };
std::cout << f(3.0) << std::endl;
```

In the square brackets, we have the capture list, in the round brackets we have the argument list, and in the curly brackets we have the function body. The capture list can be used to capture variables from the surrounding scope. For example, we can do

In classes, there can be STATIC members. It is a common variable for all the objects of that same class. The variable can then also be accessed with `ClassName::staticMember`. Note that all static members must be defined outside the class.

If we want a const function to modify variables we need to use the MUTABLE keyword. This is useful for caching, for example.

Lec 5

For classes, there exist special member functions such as the default constructor and the destructor. The most important are

```
C() // default constructor
~C() // destructor
C(C const&) // copy constructor
C& operator=(C const&) // copy assignment
    operator
C(C&& c) // move constructor
C& operator=(C&& c) // move assignment operator
```

If no constructor is defined, a default one is generated by the compiler. The destructor is called automatically, if an object goes out of scope. The guaranteed destruction is ideal to handle resources, (RAII). The copy constructor does a memberwise copy of the object if generated automatically. The copy assignment operator is used to assign one object to another. Note, the copy constructor is used when doing `C c2 = c1;` and the copy assignment operator is used when doing `c2 = c1;`. By convention, the copy assignment operator returns a reference to the object being assigned to, so that assignment can be chained. The copy assignment should also handle self-assignment, i.e. `c1 = c1;`. It also does shallow copies.

The move constructor and assignment operator are used to transfer resources from one object to another. The move constructor is useful to avoid unnecessary copies, which is the main issue with modern computers.

In C++ there exist different value categories, including: Rvalues, lvalues, xvalues, prvalues, glvalues, ...

Since C++11 one is also allowed to explicitly declare the default constructor and destructor, as well as the copy and move constructors and assignment operators. This is done by using the `= default` specifier. For example, one can write

```
class C {
public:
    C() = default; // default constructor
    ~C() = default; // destructor
    C(C const&) = delete; // copy constructor
};
```

This example shows that the default constructor and destructor are explicitly declared, while the copy constructor is deleted.

As a guideline, one can use the rule of three, rule of five, or rule of zero. The rule of three states that if a class requires a user-defined destructor, copy constructor, or copy

assignment operator, it likely requires all three. The rule of five extends this to include the move constructor and move assignment operator. The rule of zero suggests that if a class does not manage resources directly, it should not define any of these special member functions and rely on the compiler-generated ones.

When overloading a compund assignment operator like `+=`, it is typically implemented as a member function. It should always return a reference to itself. Binary operators should be implemented as non-member functions[1].

To avoid implicit conversions in for example the constructor, one can use the `explicit` keyword. This prevents the compiler from using that constructor for implicit conversions and copy-initialization. For example, consider the following class:

```
class C {
public:
    explicit C(int x) : x_(x) {}
private:
    int x_;
};
```

This can also be used if one has a conversion operator, to prevent implicit conversions.

If we want to have classes that can have different types. we can use class templates. For example we can define a class template for a pair of values as follows:

```
template <typename T>
class Point {
  public:
    using coord_t = T;
    //..
    coord_t abs() const;
    // ...
  private:
    coord_t x_;
    coord_t y_;
};

template <typename T>
typename Point<T>::coord_t Point<T>::abs() const
    {
    return std::sqrt(x_*x_ + y_*y_);
}
```

Template metaprogramming is a family of techniques to create new types and compute values at compile time. A metafunction is a struct with zero+ template parameters and zero(+) return types or values.

```
template <int X>
struct ReturnValue {
    static constexpr int value = X;
};
```

Why is this useful? Recall our minimum function where we had to make a template with 3 template parameters. We can use template metaprogramming to compute the return type at compile time. This is called type traits.

```
template <typename T, typename U>
typename min_type<T,U>::type min(T const& a, U
    const& b) {
    return a < b ? a : b;
}
```

[1] This is to maintain symmetry.

Here, `typename` is needed to tell the compiler that `min_type<T,U>::type` is a type and not a value. In this case, `min_type` is a metafunction that computes the return type of the minimum function at compile time.

```
template <typename T> struct min_type {
    using type = T;
};
template <> struct min_type<int> {
    using type = int;
};
template <> struct min_type<double> {
    using type = int;
};
```

We cannot use auto in general, since for example if we compute averages, we could run into integer divisions. We dont want to write this for all integer types so we can do

```
template <typename T>
struct average_type {
typedef typename helper<T, std::numeric_limits<T
    >::is_integer>::type type;
};

template <typename T, bool is_integer>
struct helper { typedef T type; };

template <typename T>
struct helper<T, true> { typedef double type; };
```

Lec 6

When documenting a function template, one should in addition to the usual documentation, also provide the concept requirements on the types. Note, that the complete source code of the function template must be in the header file, since the compiler needs to see the complete definition of the function template in order to instantiate it for a specific type.

6 Error Handling

Suppose we want to solve an ODE with some program. Suppose we for example want to use Newton's method to solve a nonlinear equation. We need to deal with this. One way is to print an error message and abort the program.

```
std::cout << "Error: Newton's method did not
    converge." << std::endl;
std::abort();
```

There can also be a status/error code. Usually zero means zero problems. In C++ we can also use exceptions. It allows to separate the detection of an error from the handling of the error. If C++ recognizes an exceptional situation, it throws an exception. The exception is then caught by a catch block. In nobody catches the exception, the program terminates.

```
if (n <=  0) {
  throw "n too small";
}

if ( index >= size()){
  throw std::range_error("index out of range");
}
```

If an exception is thrown, it causes the normal execution to terminate. The call stack is unwound until a catch block is found. If no catch block is found, the program terminates. In `<stdexcept>`. For example

- `std::domain_error` for domain errors, e.g. `sqrt(-1)`

- `std::invalid_argument` for invalid arguments, e.g.
- `std::length_error` for length errors, e.g. `std::vector::resize(-1)`

And many more. All have a member function `what()` that returns a string with an error message. To catch an exception, we use a `try` block and a `catch` block. For example

```
try {
  // .. try this
} catch (std::exception& e) {
  std::cout << "Error: " << e.what() << std::endl;
} catch (...) {
  std::cout << "Unknown error" << std::endl;
  // May also rethrow the exception
}
```

Some things that should not throw exceptions include destructors.

In order not to produce errors, we should have software tests. A popular method for this is UNIT TESTING. It is important to automatically run the tests. In this course we will use CTest. As a simple example we want to test if a Fibonacci function works correctly. For example, we know that $F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_{42} = 267914296$. In Cmake we can use CTest with the commands

```
include(CTest)
add_subdirectory(src)
if(BUILD_TESTING)
  add_subdirectory(tests)
endif()
```

The tests are then written in a separate file, e.g. `test_fibonacci.cpp`. It essentially consists of a main function that calls the functions to be tested and checks if the results are correct. Else it throws an exception, which is not caught. We can then add a test in CMake with

```
add_executable(test_fibonacci test_fibonacci.cpp)
target_link_libraries(test_fibonacci fibonacci)
add_test(NAME test_fibonacci COMMAND
    test_fibonacci)
```

The test is then started with `make test`.

Another framework is catch2. We write an analogue cmake file and a test file, e.g. `test_fibonacci_catch2.cpp`. The test file looks like

```
#include "fibonacci.hpp"

#define CATCH_CONFIG_MAIN

#include <catch2/catch_all.hpp>

TEST_CASE("Fibonacci function works correctly", "
    [fibonacci]") {
    REQUIRE(fibonacci(0) == 0);
    REQUIRE(fibonacci(1) == 1);
    REQUIRE(fibonacci(2) == 1);
    REQUIRE(fibonacci(3) == 2);
    REQUIRE(fibonacci(42) == 267914296);
}
```

The Cmake file inside the `tests` directory looks like

```
include(FetchContent)

FetchContent_Declare(
  catch2
  GIT_REPOSITORY https://github.com/catchorg/
    Catch2.git
  GIT_TAG v3.16.0
)

FetchContent_MakeAvailable(catch2)

add_executable(test_fibonacci_catch2
    test_fibonacci_catch2.cpp)

target_link_libraries(test_fibonacci_catch2
    PRIVATE fibonacci Catch2::Catch2WithMain)

add_test(NAME test_fibonacci_catch2 COMMAND
    test_fibonacci_catch2)
```

7 Standard Library

The standard library consist of date and time utilities. The idea is that discussions about efficiency are meaningless if you cannot measure it. The libraries for this are `<chrono>` and `<ctime>`. To get a time we can use

```
auto start = std::chrono::high_resolution_clock::
    now();
// Do something
auto end = std::chrono::high_resolution_clock::
    now();
std::chrono::duration<double> elapsed = end -
    start;
std::cout << "Elapsed time: " << elapsed.count()
    << " seconds" << std::endl;
```

Another important library is `<random>`. It provides random number generators and distributions. For example, to generate random numbers from a uniform distribution, we can use

```
std::random_device rd;  // Seed for the random
    number engine
std::mt19937 gen(rd()); // Mersenne Twister
    engine
std::uniform_real_distribution<> dis(0.0, 1.0);
    // Uniform distribution between 0 and 1
double random_number = dis(gen); // Generate a
    random number
```

One simple to implement random generator is a linear congruential generator (LCG). It is defined by the recurrence relation

$$x_{n+1} = (ax_n + c) \mod m.$$

The random library contains the random number generators (the engines) and the distribution functions. Ideally, we always want to seed the random number generator with a random seed. This helps to reproduce the simulation. The most important distributions are

- `std::uniform_int_distribution` for uniform integer distribution
- `std::uniform_real_distribution` for uniform real distribution
- `std::normal_distribution` for normal distribution
- `std::exponential_distribution` for exponential distribution
- `std::poisson_distribution` for Poisson distribution

Up to now, we added the documentation to the header files. We can also use Doxygen to generate nice documentation. Doxygen is a tool that generates documentation from annotated C++ sources. It can also generate

diagrams and graphs. To use Doxygen, we need to add comments in the source code using special tags. For example, we can use `///` or `/** ... */` to add comments. We can also use `@param` to document parameters and `@return` to document the return value.

Lec 7

We would now like to discuss some data structures and algorithms. For this, we need to analyse complexity of algorithms. It answers the question how the time needed for an algorithm scales with the problem size N.

An interesting case is the complexity of adding an element to the end of an array. Naive implementation would be to create a new array of size $N+1$, copy all elements from the old array to the new array, and then add the new element. This would take $O(N)$ time. However, if we allocate an array of size $2N$ when the array is full, we can add an element in $O(1)$ time on average. This technique is called AMORTIZED ANALYSIS. The idea is that we can afford to spend $O(N)$ time to copy the array only rarely.

In the following we will look at the standard template library (STL) which is a subset of the standard library of C++. It consists of containers (called data sequences) and algorithms. Around this, there are different things like iterators.

Simple data structures are strings and widestrings (wstring). They contain some useful functions for string manipulation. They are however not so critical for numerical simulations.

In `<utility>` we find the `pair` class. It is a simple data structure that can hold two values accesed by `first` and `second`. It can be useful to return two values from a function. We furthermore have Arrays, in the form of C arrays, vectors, valarrays and deques. There are linked lists like lists and forward lists. There also are trees such as maps sets, multimaps and multisets. The latter are sorted associative containers. Lastly we have queues and stacks.

An array is a consecutive range in memory. It has fast $\mathcal{O}(1)$ access to its elements. However, the main disadvantage is that insertion and removal at arbitrary positions is $\mathcal{O}(N)$, since all elements after the insertion/removal point have to be shifted.

The deque is a double-ended queue. It allows for fast insertion and removal at both ends.

In C++ a container is a collection of objects. All of them are listed on the cpp reference page.

One issue with our algorithms is that we may need to implement every algorithm for every data structure. This is where generic traversal comes in. Instead of going through a data structure in a special way, we want to use iterators.

In principle, for this, we need an iterator datatype, a begin and end function and a ++ operator. For an array this may look as follows

```
1  template <typename T>
2  class SArray {
3    public:
4      using size_type = std::size_t;
5      using value_type = T;
6      using iterator = T*;
7
8      iterator begin() { return data; }
9      iterator end() { return data + size; }
10 };
```

In general it is good practice to use the typedefs `size_type` and `value_type` instead of the actual types. This allows the container compatible with the standard library.

For other iterators, for example for a linked list, we may also need to implement the dereference operator `*` and the increment operator `++`. Furthermore the inequality operator `!=` is needed to compare two iterators.

Iterators exist in 5 categories. Input/Output iterators can only read/write once. Forward iterators can read/write multiple times. Bidirectional iterators can go forward and backward. Random access iterators can jump to any position in the container (in $\mathcal{O}(1)$ time). Except the output iterator, they all exist in the const version aswell.

Index

amortized analysis, 8

branches, 1

commit, 1
commit message, 1
Compiler, 2
conflict, 1

merging, 1
mutable, 5

Preprocessor, 1
pull, 1
push, 1

staging area, 1
static, 5

unit testing, 7